Are Automated Debugging Techniques really helping Programmers?

Nikhil Anand

March 25, 2026

This paper investigates the practical implications of anonymous author code review, a technique intended to reduce bias by hiding the identity of code authors during review. Motivated by prior evidence of bias in software engineering and other domains, the authors conduct a large-scale field experiment at Google involving over 5,000 code reviews by 300 engineers. The study examines key aspects of the review process, including the ability of reviewers to infer author identity, the impact on review speed and quality, and perceptions of fairness. While anonymization is theoretically expected to mitigate biases related to gender, experience, or reputation, the paper highlights that its real-world effectiveness depends on how it interacts with existing development practices and social dynamics .

The findings show that anonymous author code review has nuanced effects rather than transformative ones. Reviewers were often able to correctly guess the author's identity—especially in familiar contexts—limiting the full potential of anonymization. Despite this, the practice reduced emphasis on hierarchical relationships and encouraged more neutral evaluation behavior. Importantly, the study finds that anonymization had little to no measurable impact on review quality or objective review speed, though participants sometimes perceived it as slowing communication due to reduced opportunities for direct interaction. Overall, the paper concludes that while anonymous author review does not drastically change engineering outcomes, it offers modest benefits in promoting fairness and reducing bias, with trade-offs related to context loss and communication overhead .

1 Motivation

The motivation for this work stems from growing evidence that code review—despite being perceived as an objective, quality-driven process—is influenced by social and cognitive biases related to the identity of the author. While developers often believe that code changes are evaluated purely on technical merit, prior research shows that factors such as gender [1, 2], race [3], age [4], and even physical attractiveness [5] can influence decision-making outcomes in professional settings. In the context of software engineering, studies have demonstrated that women's contributions to open-source projects are less likely to be accepted when their gender is identifiable, suggesting that implicit bias plays a role in code review decisions [1]. These findings challenge the assumption that code review is inherently fair and motivate the need for mechanisms that can reduce the influence of such biases.

To address similar concerns in other domains, anonymization has been widely adopted as a strategy to promote fairness. For example, blind auditions in orchestras have been shown to increase the representation of women by removing visual cues about the performer [6], and double-blind peer review in academia has been found to reduce advantages associated with famous authors or prestigious institutions [7]. Inspired by these successes, the idea of anonymous author code review—where the reviewer does not know the identity of the code author—has been proposed as a potential solution to mitigate bias in software engineering [8]. However, despite its theoretical appeal and some early exploratory tools (e.g., anonymization extensions for GitHub), there is a lack of empirical evidence on how such a practice would function in real-world engineering environments. This gap is critical, as practitioners have expressed skepticism, with concerns that anonymization might hinder communication, reduce efficiency, or be impractical in collaborative development settings.

Consequently, the primary motivation of this paper is to bridge this empirical gap by systematically studying the real-world effects of anonymous author code review. The authors aim to move beyond theoretical arguments and anecdotal claims by conducting a large-scale field experiment to understand not only whether anonymization can reduce bias, but also how it impacts key engineering outcomes such as review velocity, code quality, and developer experience. Importantly, the study is not limited to fairness alone; it seeks to uncover the broader trade-offs involved in adopting anonymization in practice. By doing so, the paper positions itself at the intersection of software engineering practices and human-centered concerns, aiming to provide evidence-based guidance on whether and how anonymous author code review should be implemented in modern development workflows.

2 Methodology

2.1 Research Questions

The paper formulates six research questions to systematically evaluate the practical implications of anonymous author code review, focusing on both technical outcomes and human factors in the review process.

RQ (1) How often can reviewers guess author identities?

RQ (2) How does anonymous author code review change reviewers' velocity?

RQ (3) How does anonymous author code review change review quality?

RQ (4) What effect does anonymous author code review have on reviewers' and authors' perceptions of fairness?

RQ (5) What do engineers perceive as the biggest advantages and disadvantages of anonymous author code review?

RQ (6) What features are important to an implementation of anonymous author code review?

The first research question (**RQ (1)**) investigates whether anonymization is effective in practice by examining how often reviewers can correctly guess the identity of the author. This question is critical because prior work in double-blind peer review suggests that anonymity is effective when identities are difficult to infer [9, 10]. If reviewers can frequently deduce the author based on contextual cues such as code ownership or prior collaboration, the benefits of anonymization may be undermined. Thus, RQ1 establishes a foundational understanding of whether anonymity can realistically be preserved in software engineering environments.

The second (**RQ (2)**) and third (**RQ (3)**) research questions focus on core engineering outcomes: review velocity and review quality. RQ2 explores whether hiding author identity slows down the review process, as prior studies have identified review speed as a key concern in industrial settings [11]. Anonymity could introduce friction by limiting direct communication between reviewers and authors. RQ3 examines whether anonymization affects the quality of reviews, drawing on evidence from academic peer review where double-blind processes have been shown to yield equal or higher-quality evaluations [12, 13, 14, 15]. Together, these questions assess whether anonymous author review introduces trade-offs between fairness and efficiency or quality.

The fourth research question (**RQ (4)**) shifts focus to perceptions of fairness, a central motivation behind anonymization. Rather than attempting to measure fairness objectively, the study follows prior work in evaluating perceived fairness among participants [16, 17]. This question examines whether both reviewers and authors feel that the process becomes more equitable when identities are hidden. The fifth research question (**RQ (5)**) broadens the analysis by identifying the perceived advantages and disadvantages of anonymous author code review from the perspective of engineers. This includes understanding practical trade-offs, such as reduced bias versus loss of context or communication barriers. Finally, the sixth research question (**RQ (6)**) investigates what features are necessary for effectively implementing anonymous author code review in real-world systems, providing actionable insights for tool and process design.

2.2 Method Overview

To systematically study anonymous author code review in practice, the authors conducted a large-scale field experiment at Google involving hundreds of engineers and thousands of code reviews. The approach combines controlled experimentation with real-world deployment, enabling both causal comparison (via control and treatment groups) and ecological validity. The study is designed to answer multiple research questions by integrating quantitative metrics (e.g., review time, rollback rates) with qualitative feedback (e.g., perceptions of fairness, quality, and usability). Statistical analyses were conducted using regression models with multiple covariates to isolate the effect of anonymization from other influencing factors .

Implementation of Anonymous Author Code Review The authors implemented anonymization using a browser extension that removed author-identifying information from Google's internal code review tool, Critique. This included hiding usernames in the review interface, associated emails, and notifications. However, due to practical limitations, anonymization could not be enforced across all channels (e.g., mobile notifications, external devices, or linked bug reports), requiring participants to follow guidelines such as filtering emails and avoiding certain interfaces. The extension also provided a "break-the-glass" feature that allowed reviewers to reveal the author's identity when necessary, acknowledging that complete anonymity may not always be practical. Importantly, while reviewers experienced anonymization, authors continued to see the standard interface.

Participants and Experimental Design Participants were recruited from Google engineers who met specific criteria, including sufficient tenure, team familiarity, and recent code review activity. This ensured that participants were experienced with the review process and could provide meaningful data. A stratified sampling strategy was used to include a subset of "readability reviewers," who review code for adherence to coding standards. Volunteers were randomly assigned to either a treatment group (anonymous review) or a control group (standard review), enabling comparative analysis. The study ultimately included 300 active participants in the treatment group and 95 in the control group, generating over 5,000 reviewed changelists in the treatment condition alone.

2.3 Data Collection

2.3.1 Quantitative Measures

The study employs a mixed-methods data collection strategy, combining large-scale quantitative logging with structured qualitative feedback to capture both behavioral and perceptual aspects of anonymous author code review. Quantitatively, the researchers instrumented their browser extension and internal tooling to collect fine-grained interaction data during the review process. One key metric collected was whether and when reviewers chose to deanonymize the author, which directly informs the effectiveness of anonymization and helps answer how often anonymity is practically preserved. This logging provides an objective signal of when anonymity breaks down in real-world usage .

Another major quantitative measure is review velocity, operationalized as "active reviewing time." This metric captures the amount of time a reviewer spends actively engaging with a changelist, including reading code, commenting, and consulting related resources such as documentation or APIs. By focusing on active engagement rather than elapsed time, the study avoids noise from idle periods and provides a more accurate estimate of effort and efficiency. This measure is critical for evaluating whether anonymization introduces friction into the review process .

To assess review quality, the authors use rollback rates as a proxy. Specifically, they track whether a changelist is reverted within a long-term window (up to several months after the review). The underlying assumption is that higher-quality reviews are more likely to catch issues early, reducing the need for later rollbacks. While indirect, this measure allows the researchers to evaluate quality at scale without requiring manual inspection of code or subjective judgments. It also aligns with prior empirical work linking review outcomes to software quality.

2.3.2 Qualitative Measures

In addition to quantitative metrics, the study collects rich qualitative data through two types of structured reports. "CL reports" are completed immediately after each review and capture the reviewer's perceptions of identity guessability, velocity, quality, and fairness. These near-real-time responses reduce recall bias and reflect the reviewer's immediate experience. A second instrument, the "final report," is administered at the end of the study and captures more reflective feedback, including perceived advantages, disadvantages, and desired features of anonymized review systems. Together, these qualitative instruments provide insight into the human and experiential dimensions of the process that cannot be inferred from logs alone .

Finally, the study also gathers fairness perceptions from authors of the reviewed changelists. Importantly, authors were not informed whether their code was reviewed under anonymous conditions, reducing bias in their responses. This design allows the researchers to compare perceived fairness across conditions while minimizing expectancy effects. Overall, the data collection approach is comprehensive, triangulating across behavioral logs, self-reports, and multiple stakeholder perspectives to build a holistic understanding of anonymous author code review.

2.4 Data Analysis

The analysis phase relies on a combination of statistical modeling and qualitative coding to interpret the collected data. The authors primarily use regression-based techniques to isolate the causal effects of anonymization while controlling for confounding variables. Two main types of models are employed: review-level regressions (RLR), which analyze individual code review instances, and participant-level regressions (PLR), which aggregate behavior at the reviewer level. This dual approach allows the study to capture both fine-grained and broader patterns in the data .

A key strength of the analysis is the inclusion of extensive control variables. These include reviewer characteristics (e.g., tenure, seniority, role), author characteristics, and properties of the changelist (e.g., size, number of reviewers, type of change). The models also account for the prior relationship between reviewer and author (e.g., insider vs. outsider), which is known to influence review dynamics. By incorporating these covariates, the analysis mitigates confounding effects and ensures that observed differences between control and treatment groups can be more confidently attributed to anonymization rather than underlying differences in context .

The regression models also include random effects to account for individual variability among reviewers. This is important because different engineers may have distinct reviewing styles, speeds, or biases. By modeling reviewer identity as a random effect, the analysis captures this heterogeneity and avoids overgeneralizing from individual behavior. Statistical significance is evaluated using a standard alpha threshold (0.05), and model fit is reported using metrics such as adjusted (R^2) or pseudo-(R^2), depending on the type of regression used.

For qualitative data, the authors perform thematic coding of open-ended responses from participants. Responses are categorized into recurring themes (e.g., benefits, drawbacks, reasons for deanonymization), allowing the researchers to quantify and interpret common patterns in subjective experiences. While coding is primarily conducted by a single researcher—introducing potential subjectivity—the large volume of responses and systematic categorization provide meaningful insights into how developers perceive anonymous author review.

Overall, the data analysis approach is methodologically rigorous and well-aligned with the study's goals. By combining controlled statistical modeling with qualitative interpretation, the authors are able to not only measure the effects of anonymization but also explain the mechanisms behind those effects. This integrated approach strengthens the validity of the findings and enables a nuanced understanding of both technical outcomes and human factors in code review.

3 Results

3.1 Participation and Representativeness

Since participation in the study was voluntary, the authors first analyze who chose to participate and whether any systematic biases exist. They find that engineers with "readability" certification were significantly more likely to volunteer, while more experienced engineers (with longer tenure) were less likely to participate. Additionally, some regional differences were observed (e.g., higher participation from certain geographic regions), though most other factors did not show statistically significant effects. This suggests that while the sample is reasonably diverse, it is not perfectly representative and may slightly overrepresent certain subgroups.

The authors then examine whether participants consistently reported on all the code reviews they performed during the study period. Although participants were instructed to report every changelist (CL), compliance was not perfect. However, the reporting rates between the control and treatment groups were similar and not statistically different, indicating no systematic bias introduced by the experimental condition itself. That said, other factors—such as the size of the changelist, the number of reviewers, or whether the reviewer had prior familiarity with the author—did influence the likelihood of reporting. This reinforces the need for regression models later in the analysis to control for such confounding variables.

Another aspect of representativeness concerns how typical the reviewed changelists were compared to participants' normal work. The majority of participants in both groups reported that the changelists they reviewed during the study were either "very typical" or "somewhat typical" of their usual work. The treatment group reported slightly higher "very typical" ratings than the control group, suggesting that the anonymization process did not significantly disrupt the nature of the work being reviewed. This supports the ecological validity of the experiment, indicating that the findings are grounded in realistic development scenarios.

Finally, the authors analyze the structure of code reviews themselves, particularly the number of reviewers per changelist and the overlap between participants. Most changelists were reviewed by one or two reviewers, which aligns with standard industry practices. Only a very small fraction of reviews involved multiple study participants or a mix of control and treatment participants, minimizing the risk of cross-contamination between experimental conditions. Overall, this section concludes that while there are some participation biases inherent to voluntary studies, the dataset is sufficiently representative and well-controlled to support meaningful analysis of the research questions.

3.2 RQ1: Ability to Guess Author Identity

The results show that anonymization is often ineffective in typical development settings. In normal code reviews, reviewers were able to correctly guess the author in a large majority of cases, while uncertainty was relatively low. In contrast, in readability reviews—where reviewers are less familiar with authors—reviewers were uncertain about the author's identity most of the time. This highlights that anonymization works better in low-context scenarios but breaks down in environments with strong familiarity and ownership patterns .

Reviewers primarily inferred identities using contextual cues, such as the part of the codebase being modified, programming style, or ongoing work. Communication outside the tool (e.g., prior discussions) and limitations in the anonymization system also contributed to identity leakage. In some cases, reviewers explicitly chose to reveal the author to gain necessary context or communicate more effectively. Overall, RQ1 concludes that anonymity is difficult to preserve in collaborative, context-rich software development workflows.

3.3 RQ2: Impact on Review Velocity

From a subjective perspective, many participants believed that anonymization slowed down reviews or introduced friction, mainly due to reduced communication and lack of contextual information. While

most reported no change in individual reviews, a significant portion expected negative effects on overall engineering velocity, indicating a perception that anonymity makes the process less efficient .

However, objective analysis using active reviewing time shows no significant impact of anonymization on review speed. Both control and treatment groups experienced similar changes in review time, suggesting that observed slowdowns were likely due to external factors (e.g., study effects) rather than anonymization itself. Thus, RQ2 highlights a gap between perception and reality: developers feel that anonymization slows them down, but measurable effects are minimal.

3.4 RQ3: Impact on Review Quality

Participants generally perceived that anonymization had no significant effect on review quality, with some indicating slight improvements due to increased focus on the code rather than the author. Few participants reported negative effects, and overall sentiment leaned toward neutrality or modest benefit.

Objective analysis using rollback rates supports this perception, showing no significant difference in quality between anonymous and non-anonymous reviews. This indicates that anonymization does not harm the effectiveness of code reviews in identifying issues or maintaining code correctness. Overall, RQ3 concludes that anonymous author code review preserves quality while potentially encouraging more objective evaluation behavior.

3.5 RQ4: Effect on Perceived Fairness

Anonymous author code review shows a largely neutral to mildly positive impact on perceived fairness. From the reviewer perspective, most participants reported no change in fairness during individual reviews, but a meaningful subset perceived improvements due to reduced influence of identity-based biases. In post-study reflections, this positive perception became stronger, suggesting that while the immediate effect may be subtle, developers conceptually associate anonymization with fairer evaluation practices .

From the author perspective, fairness perceptions remained largely unchanged across anonymous and non-anonymous conditions. Since authors were unaware of whether anonymization was applied, this result is less likely to be biased by expectations. An exception appears in readability reviews, where control reviews were perceived as more fair, though further analysis suggests this is due to unusually high control ratings rather than a negative effect of anonymization itself. Overall, anonymization does not significantly shift fairness perceptions but aligns with developers' expectations of improved impartiality.

3.6 RQ5: Perceived Advantages and Disadvantages

Participants identified fairness and objectivity as the primary advantages of anonymization. Reviewers reported focusing more on the code itself rather than the author, leading to more careful and unbiased evaluations. Some also noted improved critical feedback and reduced influence of hierarchy or prior relationships. However, many participants observed little benefit in practice because they could still infer author identity through contextual cues, limiting the effectiveness of anonymization in real-world settings.

The main disadvantages center around loss of context and communication barriers. Reviewers rely on knowledge of the author (e.g., expertise, intent, ownership) to interpret changes efficiently, and anonymization removes this signal. This often led to slower reviews, difficulty in decision-making, and reduced ability to initiate quick interactions. Additional drawbacks include tool limitations, inconvenience from over-redaction, and reduced awareness of team activity. Overall, RQ5 highlights a trade-off: anonymization can reduce bias but at the cost of efficiency and collaboration.

3.7 RQ6: Important System Features

The results emphasize that strict anonymity is impractical, and successful implementations require flexibility. The most critical feature identified is the ability to "break the glass" and reveal the author when necessary. This allows reviewers to recover essential context in situations where identity is important, balancing fairness with usability.

Participants also favored partial anonymization, such as revealing non-sensitive contextual signals (e.g., team or expertise) while hiding identity. Additionally, flexible adoption models—such as opt-in usage at the team or individual level—and delayed identity revelation (e.g., after approval) were seen as useful. Overall, the findings suggest that effective systems should not enforce complete anonymity but instead provide configurable mechanisms that preserve workflow efficiency while reducing bias.

4 Limitations

The authors acknowledge several limitations related to the experimental design and execution, particularly concerning the incomplete nature of anonymization. Despite the use of a browser extension to hide author identities within the code review tool, reviewers could still infer identities through various external channels such as email notifications, bug tracking systems, or prior communication. In addition, contextual cues such as code ownership, familiarity with specific components, or ongoing tasks often enabled reviewers to correctly guess the author. This limitation directly affects the internal validity of the study, as the treatment condition does not represent a fully anonymized environment. As a result, the measured effects of anonymization may underestimate its potential impact in a more controlled or isolated setting.

Another important limitation arises from the voluntary participation and representativeness of the sample. Since engineers self-selected into the study, the participant pool may not fully reflect the broader engineering population at Google. For example, the study found that certain groups, such as readability reviewers, were more likely to participate, while more experienced engineers were less likely to volunteer. This introduces potential selection bias, which may influence both behavioral and perceptual outcomes. Additionally, not all participants consistently submitted reports for every code review they conducted, leading to incomplete data coverage. Although the authors attempt to control for these factors in their statistical models, the possibility of residual bias remains.

The study also relies on proxy and self-reported measures, which introduce additional sources of uncertainty. Review quality is approximated using rollback rates, which may not fully capture the nuances of code quality or the effectiveness of a review. Rollbacks can occur for reasons unrelated to review quality, and conversely, some issues may not result in a rollback. Similarly, qualitative measures such as perceived fairness, velocity, and quality are based on participant self-reports, which are subject to biases such as recall bias or social desirability bias. While collecting both objective and subjective data strengthens the analysis, these measures still have inherent limitations in accurately reflecting underlying constructs.

Finally, the authors highlight limitations related to generalizability and experimental context. The study is conducted within a single large organization with specific tools, workflows, and cultural practices, which may not generalize to other companies or open-source environments. The experimental setup itself may also introduce behavioral changes, such as participants being more cautious or attentive because they know they are being studied (a form of observer effect). Additionally, the relatively short duration of the experiment may not capture long-term adaptations to anonymization, such as changes in collaboration patterns or communication strategies. Together, these factors suggest that while the findings are informative, they should be interpreted with caution when considering broader adoption of anonymous author code review.

5 Implications

The discussion synthesizes the findings by emphasizing that anonymous author code review produces nuanced and context-dependent effects rather than fundamentally transforming the review process. A central takeaway is that anonymity is difficult to achieve in practice due to strong contextual signals embedded in software development workflows. Reviewers frequently rely on knowledge of code ownership, team structures, and prior interactions, which enables them to infer author identity even when explicit identifiers are removed. As a result, the potential benefits of anonymization—particularly in reducing bias—are inherently limited in environments where familiarity and collaboration are high. This aligns with prior work on anonymization and double-blind review, which shows that anonymity is most effective when contextual cues are minimal [10, 9].

At the same time, the discussion highlights a gap between perception and measurable outcomes. While developers often believe that anonymization slows down reviews and introduces friction, the empirical results show no significant impact on objective review velocity or quality. This suggests that concerns about efficiency may be more perceptual than real, potentially influenced by changes in workflow or discomfort with reduced context. Similarly, although fairness perceptions do not change dramatically in the short term, participants broadly agree that anonymization promotes more objective evaluation in principle. These findings reflect broader insights from peer review research, where anonymization does not always lead to large measurable differences but is still valued for its role in mitigating bias [7, 16].

The authors also discuss the inherent trade-offs between fairness and practicality. Removing author identity can reduce bias and encourage more careful evaluation, but it also removes useful contextual information that reviewers depend on for efficient decision-making and communication. This tension suggests that fully anonymous systems may not be suitable for all scenarios. Instead, the discussion advocates for flexible approaches, such as partial anonymization or selective identity disclosure, which can balance the benefits of reduced bias with the need for context and collaboration. This perspective reinforces the idea that anonymization should be treated as a configurable tool rather than a one-size-fits-all solution.

Finally, the discussion situates the findings within the broader landscape of software engineering practices and human factors. The results suggest that social dynamics, familiarity, and communication patterns play a significant role in code review, often outweighing the effects of structural interventions like anonymization. The authors argue that future work should focus on designing systems that better support both fairness and usability, as well as exploring contexts where anonymization may be more effective, such as cross-team or low-familiarity reviews. Overall, the discussion underscores that while anonymous author code review has potential benefits, its impact is constrained by the deeply social nature of software development.

References

[1] Josh Terrell et al. “Gender differences and bias in open source: Pull request acceptance of women versus men”. In: *PeerJ Computer Science* 3 (2017), e111.

[2] H Kristen Davison and Michael J Burke. “Sex discrimination in simulated employment contexts: A meta-analytic investigation”. In: *Journal of Vocational Behavior* 56.2 (2000), pp. 225–248.

[3] Jeffrey H Greenhaus, Saroj Parasuraman, and Wayne M Wormley. “Effects of race on organizational experiences, job performance evaluations, and career outcomes”. In: *Academy of Management Journal* 33.1 (1990), pp. 64–86.

[4] Robert A Gordon and Richard D Arvey. “Age bias in laboratory and field settings: A meta-analytic investigation”. In: *Journal of Applied Social Psychology* 34.3 (2004), pp. 468–492.

[5] Megumi Hosoda, Eugene F Stone-Romero, and Gwen Coats. “The effects of physical attractiveness on job-related outcomes: A meta-analysis of experimental studies”. In: *Personnel Psychology* 56.2 (2003), pp. 431–462.

[6] Claudia Goldin and Cecilia Rouse. “Orchestrating impartiality: The impact of “blind” auditions on female musicians”. In: *American Economic Review* 90.4 (2000), pp. 715–741.

[7] Andrew Tomkins, Min Zhang, and William D Heavlin. “Reviewer bias in single- versus double-blind peer review”. In: *Proceedings of the National Academy of Sciences* 114.48 (2017), pp. 12708–12713.

[8] Ji Young Kim, Gráinne M Fitzsimons, and Aaron C Kay. “Lean in messages increase attributions of women’s responsibility for gender inequality”. In: *Journal of Personality and Social Psychology* 115.6 (2018), p. 974.

[9] Stephen J Ceci and Douglas Peters. “How blind is blind review?” In: *American Psychologist* 39.12 (1984), pp. 1491–1494.

[10] Claire Le Goues et al. “Effectiveness of anonymization in double-blind review”. In: *Communications of the ACM* 61.6 (2018), pp. 30–33.

[11] Caitlin Sadowski et al. “Modern code review: A case study at Google”. In: *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*. 2018, pp. 181–190.

[12] Murad Alam et al. “Blinded vs. unblinded peer review of manuscripts submitted to a dermatology journal”. In: *British Journal of Dermatology* 165.3 (2011), pp. 563–567.
[13] Mary Fisher, Stephen B Friedman, and Bernard Strauss. “The effects of blinding on acceptance of research papers by peer review”. In: *JAMA* 272.2 (1994), pp. 143–146.
[14] Amy C Justice et al. “Does masking author identity improve peer review quality?” In: *JAMA* 280.3 (1998), pp. 240–242.
[15] Susan van Rooyen et al. “Effect of blinding and unmasking on the quality of peer review”. In: *JAMA* 280.3 (1998), pp. 234–237.
[16] Robert A McNutt et al. “The effects of blinding on the quality of peer review”. In: *JAMA* 263.10 (1990), pp. 1371–1376.
[17] Mark Ware. “Peer review in scholarly journals: Perspective of the scholarly community”. In: *Information Services & Use* 28.2 (2008), pp. 109–112.